# CAUSAL MACHINE LEARNING FOR ESTIMATING THE EFFECT OF DISCOUNTS ON CUSTOMER PURCHASES

M.Sc. Statistics and Computing

**Anshu Yadav**

# ABSTRACT

Predictive models can show which customers are likely to buy, but they cannot show whether a discount is what causes the purchase. This project estimates the causal effect of discounts on customer purchases and examines how the estimates can guide discount targeting. A synthetic dataset of 10,000 customers was generated with a known true average treatment effect (ATE) of 0.0150, equal to 1.50 percentage points. Discounts were assigned with a probability that depends on average order value, which creates confounding.

A naive comparison of discounted and non-discounted customers gave an ATE of 0.0209, an overstatement of 39.10%. A causal forest estimated the ATE as 0.0102 and reduced the absolute error from 0.0059 to 0.0048, although its individual-level estimates remained noisy (RMSE 0.0201, correlation 0.1755). A two-model uplift approach with 5-fold cross-fitting ranked customers weakly, with a correlation of −0.0591 with the true effects.

Statistical validation using a bootstrap (95% interval [0.0049, 0.0359]), a hypothesis test ($p$ = 0.011029) and a placebo test supported the observed difference between the groups, although these tests concern the observed difference and not the causal estimator. Targeting the top 30% of customers by predicted uplift produced more incremental revenue than random targeting (15,368.59 against 12,671.77) but slightly fewer incremental purchases (44.48 against 45.30), because the selected customers spent more per order. The true average effect was almost identical in every targeted group, so the experiment does not show that uplift targeting improves on random targeting. The ROI of 900% follows from the cost definition and does not distinguish the strategies. Across ten random seeds the naive estimate averaged 0.0260 against a true effect of 0.0150, overstating it in nine cases. The study concludes that causal methods improve the estimate of the average effect, while reliable individual-level targeting needs better estimators and experimental validation.

**Keywords:** *causal inference, confounding, causal forest, uplift modelling, heterogeneous treatment effects, discount targeting*

# TABLE OF CONTENTS

Abstract....i
List of Figures....v
List of Tables....vi
List of Abbreviations....vii
1. INTRODUCTION....1
1.1 Background....1
1.2 Problem Statement....1
1.3 Motivation....1
1.4 Significance of the Study....1
2. OBJECTIVES OF THE STUDY....2
2.1 Primary Objective....2
2.2 Secondary Objectives....2
3. RESEARCH QUESTIONS AND HYPOTHESES....3
3.1 Research Questions....3
3.2 Hypotheses....3
4. CONCEPTUAL BACKGROUND....4
4.1 Causal Inference....4
4.2 Treatment and Control....4
4.3 Average Treatment Effect (ATE)....4
4.4 Conditional Average Treatment Effect (CATE)....4
4.5 Confounding....4
4.6 Uplift Modeling....5
5. DATA AND METHODOLOGY....6
5.1 Overall Research Workflow....6
5.2 Synthetic Dataset....6
5.3 Variables....6
5.4 Treatment Assignment....7
5.5 Outcome Generation....7
5.6 Software and Libraries....8
5.7 Evaluation Metrics....8
6. STEP 1: SYNTHETIC DATA GENERATION....9
6.1 Data Generation Procedure....9
6.2 Descriptive Results....9
6.3 Confounding Check....9
6.4 True Treatment Effect....9
6.5 Interpretation....9
7. STEP 2: NAIVE TREATMENT EFFECT ESTIMATION....11
7.1 Method....11
7.2 Results....11
7.3 Interpretation....11
8. STEP 3: CAUSAL FOREST....12
8.1 Methodology....12
8.2 ATE Results....12
8.3 Error Comparison....12
8.4 Heterogeneous Effects....12

8.5 Interpretation....13
9. STEP 4: UPLIFT MODELING....14
9.1 Two-Model Approach....14
9.2 Cross-Fitting....14
9.3 Uplift Results....14
9.4 Targeting Performance....14
9.5 Interpretation....15
10. STEP 5: STATISTICAL VALIDATION....16
10.1 Bootstrap Confidence Interval....16
10.2 Hypothesis Test....16
10.3 Placebo Test....16
10.4 Interpretation....16
11. STEP 6: ROI AND TARGETING STRATEGY....17
11.1 Business Assumptions....17
11.2 Targeting Results....17
11.3 Incremental Revenue....17
11.4 ROI Interpretation....17
12. STEP 7: ROBUSTNESS ANALYSIS....19
12.1 Random Seed Analysis....19
12.2 Results....19
12.3 Interpretation....19
13. OVERALL RESULTS AND VISUALIZATION....21
13.1 Summary of Major Results....21
13.2 Treatment Effect Visualizations....21
13.3 Uplift Visualizations....22
13.4 Bootstrap and Placebo Figures....23
13.5 ROI and Revenue Figures....24
14. DISCUSSION....25
14.1 Major Findings....25
14.2 Statistical Discussion....25
14.3 Business Discussion....25
14.4 Comparison of Methods....25
15. LIMITATIONS....27
15.1 Synthetic Data....27
15.2 Individual-Level Effect Estimation....27
15.3 ROI Assumption....27
15.4 Causal Interpretation....27
16. CONCLUSION....28
16.1 Summary of the Study....28
16.2 Key Contribution....28
16.3 Final Interpretation....28
17. FUTURE SCOPE....29
17.1 Real-World Dataset....29
17.2 Experimental Validation....29
17.3 Advanced Uplift Methods....29
17.4 Improved ROI Modeling....29
17.5 Practical Deployment....29
18. REFERENCES....30

18.1 Academic References.....30
18.2 Software Documentation.....30
18.3 Dataset and Method References.....30
19. APPENDIX.....31
Appendix A: Python Code.....31
Appendix B: Detailed Output.....52
Appendix C: Additional Tables and Figures.....52

# LIST OF FIGURES

Figure 1. Confounding of the discount effect by average order value.....5
Figure 2. Overall research workflow.....6
Figure 3. Comparison of true, naive and causal forest estimates of the ATE.....22
Figure 4. True individual treatment effect against predicted uplift.....22
Figure 5. True average treatment effect by share of customers targeted.....23
Figure 6. Distribution of predicted uplift.....23
Figure 7. Bootstrap and placebo intervals for the estimated effect.....23
Figure 8. Incremental revenue and expected incremental purchases by targeting strategy.....24
Figure 9. Bootstrap distribution of the ATE.....53
Figure 10. Placebo distribution of the estimated effect.....53

# LIST OF TABLES


Table 1. Variables in the synthetic dataset.....6
Table 2. Descriptive summary of the synthetic dataset.....9
Table 3. Naive estimate compared with the true ATE.....11
Table 4. Average treatment effect estimates.....12
Table 5. Heterogeneous treatment effect estimates from the causal forest.....12
Table 6. Performance of the two-model uplift approach.....14
Table 7. Targeting performance of the uplift ranking.....15
Table 8. Summary of statistical validation results.....16
Table 9. Incremental purchases, revenue and return on investment by strategy.....17
Table 10. Naive ATE and bias for each random seed (true ATE = 0.0150).....19
Table 11. Summary of the naive ATE across the ten seeds.....19
Table 12. Consolidated summary of results.....21
Table 13. Strengths and weaknesses of the methods compared.....25
Table 14. Ten highest causal forest estimates of the individual treatment effect.....52

# LIST OF ABBREVIATIONS

| | |
|---|---|
| **ATE** | Average Treatment Effect |
| **CATE** | Conditional Average Treatment Effect |
| **CI** | Confidence Interval |
| **ML** | Machine Learning |
| **RF** | Random Forest |
| **ROI** | Return on Investment |
| **RMSE** | Root Mean Squared Error |

# 1. INTRODUCTION

## 1.1 Background

Organisations increasingly base their decisions on data. Predictive analytics, which uses machine learning to forecast outcomes such as whether a customer will buy a product, is now a routine part of this practice. Prediction, however, answers a different question from the one a decision-maker often needs answered. A model can say that a customer is likely to purchase, but it cannot say whether a discount offered to that customer is what causes the purchase.

The question of cause is the subject of causal inference. In the potential-outcomes framework (Rubin, 1974; Imbens & Rubin, 2015), the effect of an intervention on a customer is the difference between the outcome with the intervention and the outcome without it. Only one of these outcomes is ever observed for a given customer, so the effect must be estimated with suitable assumptions and methods.

## 1.2 Problem Statement

This study estimates whether providing a discount causes an additional purchase. It aims to measure the size of this incremental effect, to separate it from differences that arise only because of which customers receive discounts, and to use the estimates to guide who should be offered a discount.

## 1.3 Motivation

Discounts are rarely assigned at random. They are often directed to customers with particular characteristics, such as those who spend more per order. Purchase rates then differ between customers who received a discount and those who did not, partly because of the discount and partly because of the pre-existing differences between the two groups. Observed differences may therefore be misleading as a measure of what the discount achieves.

Moreover, the effect of a discount is unlikely to be the same for every customer. If some customers respond more than others, a business can make better use of a limited discount budget by directing it to the customers who respond most. This requires estimates of the incremental effect for different customers and not only predictions of purchase.

## 1.4 Significance of the Study

The study is relevant to statistics, machine learning and business analytics. It provides a setting with a known true effect, in which the bias of a naive estimator and the accuracy of causal machine learning methods can be measured directly. It contrasts predictive modelling with the estimation of incremental effects, and it shows how effect estimates can be turned into a targeting strategy and assessed in terms of incremental revenue.

# 2. OBJECTIVES OF THE STUDY

## 2.1 Primary Objective

The primary objective is to estimate the causal, or incremental, effect of discounts on the probability that a customer makes a purchase.

## 2.2 Secondary Objectives

The secondary objectives are as follows:

- To compare naive estimation with causal machine learning.
- To estimate heterogeneous treatment effects across customers.
- To build an uplift model for targeting discounts.
- To validate the results statistically.
- To evaluate targeting and its business value.
- To assess the robustness of the findings.

# 3. RESEARCH QUESTIONS AND HYPOTHESES

## 3.1 Research Questions

The study is organised around five research questions:

- **RQ1.** What is the causal effect of a discount on the probability of purchase?
- **RQ2.** Does confounding cause a naive comparison of discounted and non-discounted customers to misstate the true effect?
- **RQ3.** Can causal machine learning estimate both the average effect and the variation in effects across customers?
- **RQ4.** Does targeting customers by predicted uplift generate more incremental revenue than random targeting?
- **RQ5.** Are the findings robust to resampling, placebo tests and changes in the random seed?

## 3.2 Hypotheses

For the observed difference between customers who received a discount and those who did not, the hypotheses are stated in terms of the purchase rates $p_1$ (discount) and $p_0$ (no discount):

**$H_0$:** $p_1 = p_0$, that is, there is no difference in purchase rate between the two groups.

**$H_1$:** $p_1 \neq p_0$, that is, the purchase rates differ.

Rejecting $H_0$ shows a statistical difference between the groups. It does not by itself show that the discount caused the difference, because the groups may differ in other ways. The causal interpretation is therefore assessed separately, by comparing the estimates with the known true effect.

# 4. CONCEPTUAL BACKGROUND

## 4.1 Causal Inference

Association and causation are different. Two quantities may move together because one causes the other, because both are driven by a third factor, or by chance. Causal inference seeks to isolate the first of these. It asks what would have happened to the same customers under a different action, which is a question about counterfactual outcomes that cannot be observed directly (Pearl, 2009).

## 4.2 Treatment and Control

In this study the treatment is the offer of a discount, and the control condition is the absence of a discount. Customers who receive the discount form the treatment group, and customers who do not form the control group. The outcome is a binary indicator of whether the customer makes a purchase.

## 4.3 Average Treatment Effect (ATE)

Let $Y(1)$ and $Y(0)$ be the outcomes a customer would have with and without the discount. The average treatment effect is the mean of the difference between them across all customers:

$$\text{ATE} = \text{E}[\, Y(1) - Y(0) \,] \tag{1}$$

In business terms, the ATE is the average number of additional purchases per customer that the discount produces. An ATE of 0.0150, for example, means that the discount raises the probability of purchase by 1.50 percentage points on average.

## 4.4 Conditional Average Treatment Effect (CATE)

The effect of a discount may differ across customers. The conditional average treatment effect is the average effect for customers with a given set of characteristics $X = x$:

$$\text{CATE: } \tau(x) = \text{E}[\, Y(1) - Y(0) \mid X = x \,] \tag{2}$$

It describes effects at the level of individuals or subgroups, and it is the quantity needed for personalised targeting.

## 4.5 Confounding

A confounder is a variable that influences both whether a customer receives the treatment and the outcome. In the simulated experiment of this study, average order value plays this role: customers with higher order values are more likely to receive a discount, and they also tend to purchase more often. Figure 1 shows this structure. Because of it, the observed difference in purchase rates combines the effect of the discount with the effect of order value, and it overstates the benefit of the discount. Methods that adjust for characteristics related to

treatment assignment, for example through the propensity score (Rosenbaum & Rubin, 1983), are designed to address this problem.

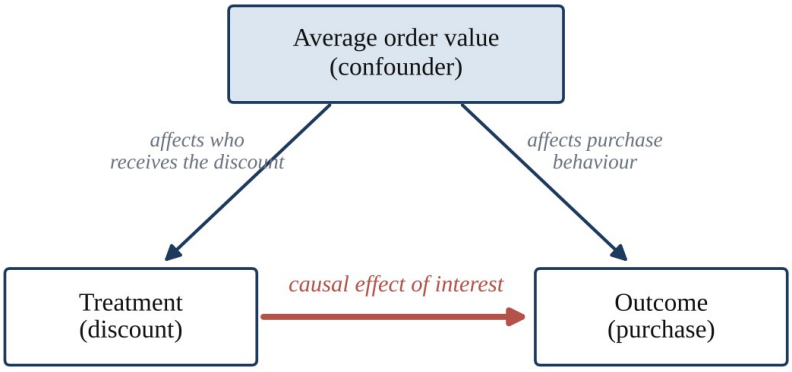



**Figure 1.** Confounding of the discount effect by average order value.

## 4.6 Uplift Modeling

Uplift modelling predicts the incremental response of a customer to a treatment, that is, the change in purchase probability caused by the discount. It differs from a conventional purchase model, which predicts the probability of purchase regardless of the cause. Uplift estimates are used for targeting: discounts are offered to the customers with the highest predicted uplift, so that the budget is spent where it adds the most purchases.

# 5. DATA AND METHODOLOGY

## 5.1 Overall Research Workflow

The analysis proceeds in seven steps, shown in Figure 2: data generation, naive estimation, causal forest estimation, uplift modelling, statistical validation, return-on-investment analysis and a robustness analysis. Each step is the subject of one chapter, from Chapter 6 to Chapter 12.

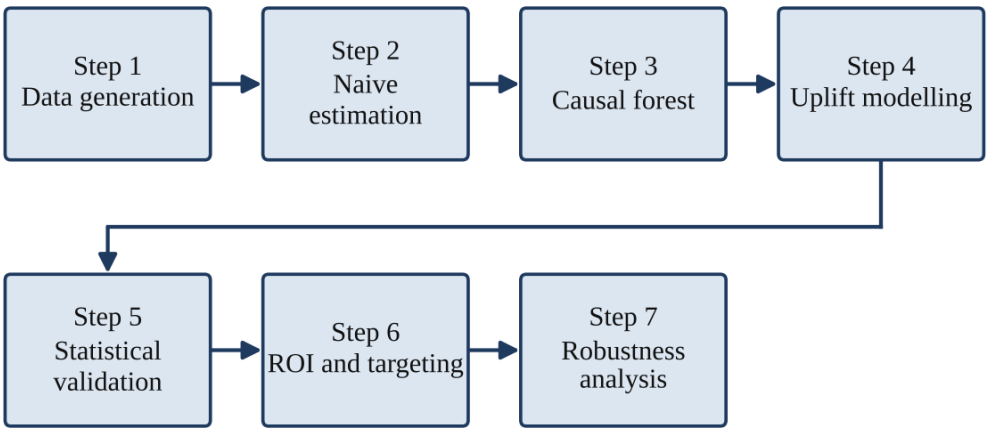



**Figure 2.** Overall research workflow.

## 5.2 Synthetic Dataset

The study uses a synthetic dataset of 10,000 customers. Because the data are simulated, the true effect of the discount on each customer is known. This is the main advantage of the design: the estimates of each method can be compared with the truth, which is impossible with real data, where the true effect is never observed. The dataset therefore serves as a benchmark for evaluating naive and causal methods.

## 5.3 Variables

Table 1 describes the variables in the dataset. The four customer characteristics were generated independently of one another.

**Table 1.** Variables in the synthetic dataset.

| Variable | Type | Description | Role |
|---|---|---|---|
| previous_purchases | Integer | Number of earlier purchases, drawn uniformly from 0 to 49 | Covariate and effect modifier |
| avg_order_value | Numeric | Typical amount spent per order, drawn uniformly between 10 and 500 | Confounder and effect modifier |

| Variable | Type | Description | Role |
|---|---|---|---|
| days_since_last_purchase | Integer | Days since the last purchase, from 1 to 364 | Covariate (does not enter the simulation) |
| email_clicked | Binary | Whether the customer clicked a marketing email; about 35% of customers click | Covariate and effect modifier |
| treatment | Binary | 1 if the customer received a discount, 0 otherwise | Treatment |
| outcome | Binary | 1 if the customer made a purchase, 0 otherwise | Outcome |

In addition, the simulation records the true individual treatment effect of each customer. This is not an observed variable. It is used only to evaluate the estimators.

## 5.4 Treatment Assignment

In the equations below, *PP*, *AOV* and *EC* denote previous_purchases, avg_order_value and email_clicked. Discount assignment is confounded by average order value. The probability that a customer receives a discount is

$$P(T = 1 \mid X) = 0.20 + 0.50 \times AOV / 500 \tag{3}$$

limited to the range 0.05 to 0.90. The probability therefore rises from about 0.21 for the lowest order values to about 0.70 for the highest, so customers who spend more per order are much more likely to be offered a discount. Treatment was then drawn at random with this probability. The other characteristics do not affect treatment assignment, so average order value is the only confounder. This mirrors the way discounts are often targeted in practice.

## 5.5 Outcome Generation

Each customer has a baseline probability of purchase without a discount:

$$b = 0.10 + 0.002\ PP + 0.00015\ AOV + 0.05\ EC \tag{4}$$

limited to the range 0.01 to 0.80. The individual treatment effect is generated from the customer's characteristics:

$$\tau_{raw} = 0.005 + 0.00015\ PP + 0.000015\ AOV + 0.002\ EC \tag{5}$$

and then multiplied by 0.015 divided by the mean of $\tau_{raw}$, so that the average true effect equals 0.015. The true individual effects then range from 0.0059 to 0.0248, with a mean and median of 0.0150. A customer's probability of purchase is $b$ without a discount and $b + \tau$ with one, and the observed purchase is drawn using the probability that matches the treatment the customer actually received.

The true effect rises with average order value (correlation 0.667 in this dataset), with previous purchases and with email clicks. Average order value is therefore both the confounder and the main source of variation in the treatment effect.

## 5.6 Software and Libraries

The analysis was carried out in Python on Google Colab. The main packages were pandas and NumPy for data handling, scikit-learn (version 1.6.1) for the random forest models, EconML (version 0.17.0) for the causal forest, SciPy for the statistical tests, and Matplotlib for visualisation. The random seed was set to 42 wherever randomness was involved.

## 5.7 Evaluation Metrics

The estimators and strategies are evaluated using the following measures:

- **ATE, bias and error:** the estimated ATE, its bias relative to the true ATE, and its absolute and relative error.
- **CATE RMSE and correlation:** the root mean squared error and the correlation between estimated and true individual effects.
- **Confidence interval, p-value and placebo interval:** measures of statistical uncertainty and of the validity of the observed difference.
- **Incremental revenue and ROI:** the additional revenue generated by a targeting strategy and its return relative to cost.

# 6. STEP 1: SYNTHETIC DATA GENERATION

## 6.1 Data Generation Procedure

Customer characteristics were generated first. Discounts were then assigned with a probability that depends on average order value. Purchase outcomes were finally generated from the baseline purchase probability and the individual treatment effect, as described in Chapter 5. Because the effects are built into the simulation, the exact ATE and CATE of the data are known.

## 6.2 Descriptive Results

Table 2 summarises the generated data. A discount was received by 45.42% of the 10,000 customers. The overall purchase rate was 21.01%, with 20.06% among customers without a discount and 22.15% among customers with a discount.

**Table 2.** Descriptive summary of the synthetic dataset.

| Quantity | Value |
|---|---|
| Number of customers | 10,000 |
| Treatment rate (discount) | 45.42% |
| Overall purchase rate | 21.01% |
| Purchase rate without discount | 20.06% |
| Purchase rate with discount | 22.15% |
| True ATE | 0.0150 (1.50 percentage points) |

## 6.3 Confounding Check

The average order value of customers who received a discount is 296.24, compared with 217.49 for those who did not, a difference of 78.75 or about 36%. The two groups were therefore not comparable before any discount was given, which confirms that confounding is present in the data.

## 6.4 True Treatment Effect

The true ATE is 0.0150, which is an increase of 1.50 percentage points in the probability of purchase. All estimators in the following chapters are judged against this value.

## 6.5 Interpretation

The controlled simulation makes it possible to evaluate each method against a known answer. This allows the bias of the naive estimator and the accuracy of the causal methods to

be measured directly. The results describe how the methods behave in a simulated setting and cannot be assumed to carry over unchanged to real customer data.

# 7. STEP 2: NAIVE TREATMENT EFFECT ESTIMATION

## 7.1 Method

The naive estimate is the observed purchase rate of treated customers minus the observed purchase rate of control customers:

$$\mathrm{ATE}_{\mathrm{naive}} = p_1 - p_0 \tag{6}$$

This estimator is valid only if treatment is assigned independently of customer characteristics, as in a randomised experiment. That condition does not hold in these data.

## 7.2 Results

**Table 3.** Naive estimate compared with the true ATE.

| Quantity | Value |
|---|---|
| True ATE | 0.0150 |
| Naive ATE | 0.0209 |
| Bias (naive − true) | 0.0059 |
| Relative error | 39.10% |

The naive estimate of 0.0209 exceeds the true effect of 0.0150 by 0.0059, a relative error of 39.10%. It equals the difference between the purchase rates of 22.15% and 20.06% reported in Table 2.

## 7.3 Interpretation

The overstatement is caused by confounding. Customers with higher average order values were more likely to receive a discount, and customers of this kind also tend to buy more often. The naive comparison therefore credits the discount with purchases that would have occurred anyway. A business relying on it would overstate the benefit of its discounts by almost 40%.

# 8. STEP 3: CAUSAL FOREST

## 8.1 Methodology

A causal forest (Wager & Athey, 2018; Athey et al., 2019) is a method for estimating both average and heterogeneous treatment effects. It adapts the random forest so that its trees split the data to find groups of customers whose treatment effects differ, rather than to predict the outcome itself. Honest estimation, in which different samples are used to choose splits and to estimate effects, limits overfitting. The method also adjusts for customer characteristics that are related to treatment assignment and to the outcome, in the spirit of double machine learning (Chernozhukov et al., 2018). It produces an estimate of the effect for each customer, and averaging these gives an estimate of the ATE.

The estimator was EconML's CausalForestDML. The outcome and treatment adjustment models were random forests with 200 trees and a minimum leaf size of 10, and the causal forest itself had 300 trees, a minimum leaf size of 10 and a maximum depth of 5.

## 8.2 ATE Results

**Table 4.** Average treatment effect estimates.

| Method | Estimated ATE | Absolute error | Relative error |
|---|---|---|---|
| True effect | 0.0150 | – | – |
| Naive estimate | 0.0209 | 0.0059 | 39.10% |
| Causal forest | 0.0102 | 0.0048 | 32.09% |

The causal forest estimate of the ATE is 0.0102, compared with a true value of 0.0150 and a naive estimate of 0.0209.

## 8.3 Error Comparison

The absolute error of the naive estimate is 0.0059 (39.10% of the true effect), whereas that of the causal forest is 0.0048 (32.09%), a reduction of about 19% in absolute error. The two errors also differ in direction: the naive estimate is too high and the causal forest estimate too low. The causal forest is therefore closer to the truth, but the improvement is modest.

## 8.4 Heterogeneous Effects

**Table 5.** Heterogeneous treatment effect estimates from the causal forest.

| Quantity | Value |
|---|---|
| Range of true individual effects | 0.0059 to 0.0248 |
| Range of estimated individual effects | −0.0751 to 0.1000 |

| Quantity | Value |
|---|---|
| CATE RMSE | 0.0201 |
| Correlation with true effects | 0.1755 |

The estimated individual effects range from −0.0751 to 0.1000, far wider than the true range of 0.0059 to 0.0248. The RMSE of 0.0201 is of the same order as the average effect itself, and the correlation with the true effects is only 0.1755.

The customers with the highest estimates show both the strength and the weakness of the method. The ten highest estimates, from 0.0887 to 0.1000, all belong to customers with average order values between about 488 and 500 and no email click. Their true effects, from 0.0205 to 0.0225, are indeed above the average of 0.0150, so the forest has found customers who respond more than average. However, it overstates the size of their effects roughly four to five times.

## 8.5 Interpretation

The causal forest produced a closer estimate of the ATE than the naive estimator, but its individual-level estimates remained noisy. The estimated effects are positively but weakly related to the true effects, and their spread, which includes negative values that do not exist in the true effects, reflects estimation noise and not genuine variation between customers. The individual estimates should therefore be used with caution.

# 9. STEP 4: UPLIFT MODELING

## 9.1 Two-Model Approach

The two-model approach (Künzel et al., 2019) fits separate outcome models for treated and control customers. The predicted uplift of a customer is the difference between the two predicted purchase probabilities:

$$\text{uplift}(x) = \mu_1(x) - \mu_0(x) \tag{7}$$

Here $\mu_1(x)$ and $\mu_0(x)$ are the predicted probabilities of purchase for a customer with characteristics $x$ with and without a discount. Customers are then ranked by predicted uplift.

## 9.2 Cross-Fitting

The predictions were produced by 5-fold cross-fitting. The data are divided into five parts. The models are trained on four parts and used to predict for the fifth, and this is repeated so that every customer receives a prediction from models that did not use that customer's own data. This reduces overfitting and information leakage. Each model is a random forest classifier with 300 trees, a maximum depth of 5 and a minimum leaf size of 30, and the folds were stratified by treatment status.

## 9.3 Uplift Results

**Table 6.** Performance of the two-model uplift approach.

| Quantity | Value |
|---|---|
| Mean predicted uplift | 0.0093 |
| True mean effect | 0.0150 |
| RMSE against true effects | 0.0413 |
| Correlation with true effects | −0.0591 |

The mean predicted uplift of 0.0093 is below the true mean of 0.0150. The RMSE of 0.0413 is large relative to the size of the effect, and the correlation of −0.0591 is close to zero.

## 9.4 Targeting Performance

Customers were ranked by predicted uplift, and the true average effect was calculated for the top 10%, 20% and so on up to 100% of customers. If the ranking were informative, the true effect would be highest in the top groups and fall towards the overall mean of 0.0150 as more customers are included. Table 7 shows the result.

**Table 7.** Targeting performance of the uplift ranking.

| Customers targeted | Number of customers | Mean predicted uplift | Mean true effect |
|---|---|---|---|
| Top 10% | 1,000 | 0.0779 | 0.0155 |
| Top 20% | 2,000 | 0.0642 | 0.0150 |
| Top 30% | 3,000 | 0.0551 | 0.0148 |
| Top 40% | 4,000 | 0.0479 | 0.0148 |
| Top 50% | 5,000 | 0.0416 | 0.0147 |
| Top 60% | 6,000 | 0.0356 | 0.0147 |
| Top 70% | 7,000 | 0.0297 | 0.0148 |
| Top 80% | 8,000 | 0.0237 | 0.0148 |
| Top 90% | 9,000 | 0.0174 | 0.0149 |
| All customers | 10,000 | 0.0093 | 0.0150 |

The predicted uplift falls steeply, from 0.0779 in the top 10% to 0.0093 for all customers, but the true effect hardly changes. The top 10% have a true average effect of 0.0155, only 0.0005 above the overall mean, and the groups from 20% to 90% are at or slightly below it. In the top 10% the predicted uplift is about five times the true effect.

## 9.5 Interpretation

The uplift model did not rank customers reliably in this experiment. The correlation between predicted and true uplift is slightly negative, and the targeting curve is almost flat: selecting customers by predicted uplift does not select customers with materially larger true effects (Figures 4 to 6). This is reported openly, because it limits what can be claimed about personalised targeting from this experiment. A likely reason is that the average effect, 1.5 percentage points, is small relative to the noise in binary purchase outcomes, so the difference between two separately fitted models is dominated by estimation error. Predicted uplift is also related to average order value (correlation 0.36), which matters for the revenue results in Chapter 11.

# 10. STEP 5: STATISTICAL VALIDATION

## 10.1 Bootstrap Confidence Interval

A bootstrap with 500 replications (Efron & Tibshirani, 1993) was used to quantify the uncertainty of the observed difference between treated and control customers. Customers were resampled with replacement, the difference in purchase rates was recomputed each time, and the 2.5th and 97.5th percentiles gave the interval. The mean bootstrap ATE was 0.0205, with a 95% confidence interval of [0.0049, 0.0359]. The interval excludes zero and contains the true ATE of 0.0150.

## 10.2 Hypothesis Test

Welch's two-sample *t*-test of the difference in purchase rates gave $t = 2.5423$ and $p = 0.011029$. The null hypothesis of no difference was therefore rejected at the 5% level of significance.

## 10.3 Placebo Test

In a placebo test, treatment labels are randomly permuted, 500 times, so that any true effect is removed. The mean placebo ATE was 0.0001 with a 95% interval of [−0.0161, 0.0172], which is centred on zero and contains it. The procedure therefore does not find an effect where none exists.

**Table 8.** Summary of statistical validation results.

| Test | Result |
|---|---|
| Bootstrap mean ATE (500 replications) | 0.0205 |
| Bootstrap 95% confidence interval | [0.0049, 0.0359] |
| Hypothesis test | $t = 2.5423$, $p = 0.011029$ |
| Decision at the 5% level | Reject $H_0$ |
| Placebo mean ATE | 0.0001 |
| Placebo 95% interval | [−0.0161, 0.0172] |

## 10.4 Interpretation

These results concern the observed difference between treated and control customers. They show that the difference is statistically distinguishable from zero and does not arise from the labelling alone. They do not test the causal forest estimator itself, and so they do not confirm that estimator. The observed difference is also affected by confounding, as Chapter 7 demonstrated.

# 11. STEP 6: ROI AND TARGETING STRATEGY

## 11.1 Business Assumptions

The return on investment analysis uses a discount rate of 10%. The expected incremental purchases from a customer equal the customer's true individual treatment effect, and the incremental revenue equals this effect multiplied by the customer's average order value. The cost of the discount is 10% of that incremental revenue. These quantities are calculated from the true effects of the selected customers, so they measure the value of the selection itself and not the accuracy of the estimated effects.

## 11.2 Targeting Results

Three strategies were compared: targeting the top 10% of customers by predicted uplift, targeting the top 30%, and targeting a random 30% as a benchmark. Table 9 gives the results.

## 11.3 Incremental Revenue

**Table 9.** Incremental purchases, revenue and return on investment by strategy.

| Strategy | Incremental purchases | Incremental revenue | Cost (10%) | Net benefit | ROI |
|---|---|---|---|---|---|
| Top 10% by uplift | 15.46 | 5,965.56 | 596.56 | 5,369.00 | 900% |
| Top 30% by uplift | 44.48 | 15,368.59 | 1,536.86 | 13,831.73 | 900% |
| Random 30% | 45.30 | 12,671.77 | 1,267.18 | 11,404.59 | 900% |

The top 30% by predicted uplift generated more incremental revenue than a random 30% (15,368.59 against 12,671.77, a gain of 2,696.82 or about 21%), but slightly fewer expected incremental purchases (44.48 against 45.30). The explanation is the order value of the selected customers. Predicted uplift is correlated with average order value, so the top 30% have a mean order value of 317.9, compared with 257.2 for the random 30%, and incremental revenue weights each customer's effect by order value. The revenue gain therefore reflects the selection of higher-spending customers and not the identification of customers who respond more to the discount. The top 10% group generated 5,965.56 from 15.46 expected incremental purchases.

## 11.4 ROI Interpretation

All three strategies show an ROI of 900%. This is a consequence of the cost definition and not evidence of performance. Because the cost of the discount is defined as 10% of incremental revenue $R$, ROI is mechanically

$$\text{ROI} = (R - 0.10\,R) / (0.10\,R) = 9 \tag{8}$$

or 900%, for every strategy. ROI therefore cannot distinguish between strategies. Taken together with the targeting curve in Chapter 9 and the purchase counts in Table 9, the evidence does not show that uplift-based targeting outperforms random targeting in this experiment.

# 12. STEP 7: ROBUSTNESS ANALYSIS

## 12.1 Random Seed Analysis

To check that the findings do not depend on one simulated dataset, the data generation and naive estimation were repeated with ten random seeds: 42, 100, 200, 300, 400, 500, 600, 700, 800 and 900. For each repetition the naive estimate was compared with the true ATE of 0.0150.

## 12.2 Results

Table 10 gives the naive estimate and its bias for each seed, and Table 11 summarises them.

**Table 10.** Naive ATE and bias for each random seed (true ATE = 0.0150).

| Seed | Naive ATE | Bias |
|---|---|---|
| 42 | 0.0209 | 0.0059 |
| 100 | 0.0151 | 0.0001 |
| 200 | 0.0323 | 0.0173 |
| 300 | 0.0301 | 0.0151 |
| 400 | 0.0361 | 0.0211 |
| 500 | 0.0233 | 0.0083 |
| 600 | 0.0304 | 0.0154 |
| 700 | 0.0117 | −0.0033 |
| 800 | 0.0265 | 0.0115 |
| 900 | 0.0336 | 0.0186 |

**Table 11.** Summary of the naive ATE across the ten seeds.

| Quantity | Value |
|---|---|
| Mean naive ATE | 0.02600 |
| Standard deviation of naive ATE | 0.00813 |
| Mean bias | 0.01100 |
| Mean absolute error | 0.01166 |
| Maximum absolute error | 0.02114 |

## 12.3 Interpretation

The naive estimate was above the true value of 0.0150 in nine of the ten seeds, and in eight of them the overstatement exceeded 0.005. Only seed 100 was essentially unbiased (bias

0.0001), and only seed 700 gave an estimate below the truth. The mean naive ATE of 0.02600 is 0.01100 above the true effect, an overstatement of about 73%. The overstatement is therefore systematic, as expected when confounding pushes the comparison in one direction.

Two further points follow. First, the main run (seed 42) was one of the milder cases: its overstatement of 39.10% is well below the average across seeds. Second, sampling variability is considerable. The standard deviation of 0.00813 is more than half the size of the true effect, and the largest absolute error, 0.02114, exceeds the true effect itself. A single dataset can therefore give a very different naive estimate from another, which is a further reason not to rely on a naive comparison.

# 13. OVERALL RESULTS AND VISUALIZATION

## 13.1 Summary of Major Results

Table 12 brings together the main results of the study.

**Table 12.** Consolidated summary of results.

| Analysis | Metric | Result |
|---|---|---|
| True effect | ATE | 0.0150 |
| Naive estimator | ATE (absolute error) | 0.0209 (0.0059) |
| Causal forest | ATE (absolute error) | 0.0102 (0.0048) |
| Causal forest | CATE RMSE; correlation | 0.0201; 0.1755 |
| Two-model uplift | Mean predicted uplift | 0.0093 |
| Two-model uplift | RMSE; correlation | 0.0413; −0.0591 |
| Uplift targeting | Mean true effect: top 10% vs all customers | 0.0155 vs 0.0150 |
| Bootstrap | Mean ATE and 95% interval | 0.0205 [0.0049, 0.0359] |
| Hypothesis test | *t*; *p* | 2.5423; 0.011029 |
| Placebo test | Mean and 95% interval | 0.0001 [−0.0161, 0.0172] |
| Targeting (top 30% vs random 30%) | Incremental purchases | 44.48 vs 45.30 |
| Targeting (top 30% vs random 30%) | Incremental revenue | 15,368.59 vs 12,671.77 |
| Robustness (ten seeds) | Mean naive ATE (SD) | 0.02600 (0.00813) |

## 13.2 Treatment Effect Visualizations

Figure 3 compares the true ATE with the naive and causal forest estimates. The naive estimate lies above the true value and the causal forest estimate lies below it.

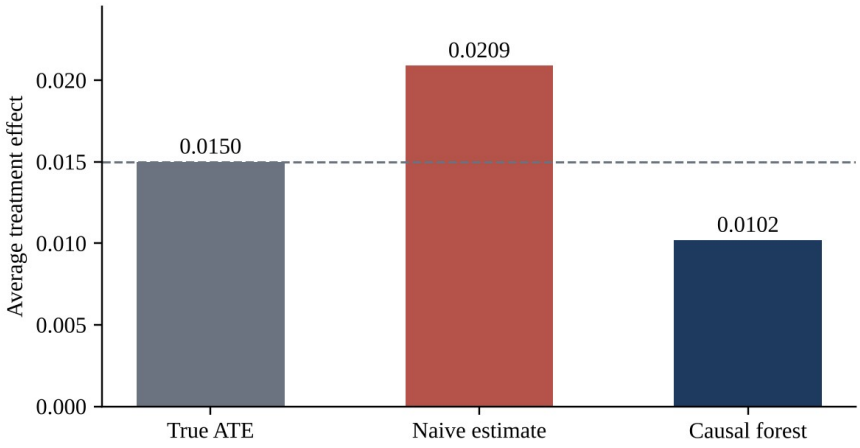



**Figure 3.** Comparison of true, naive and causal forest estimates of the ATE.

## 13.3 Uplift Visualizations

Figures 4 to 6 show the performance of the uplift model. In Figure 4, the true effects lie in a narrow band, while the predicted uplifts spread far more widely and show no relation to them. Figure 5 shows that the true average effect stays close to the overall mean however many customers are targeted. Figure 6 shows the distribution of predicted uplift, which is centred near zero and includes many negative values that do not exist in the true effects.

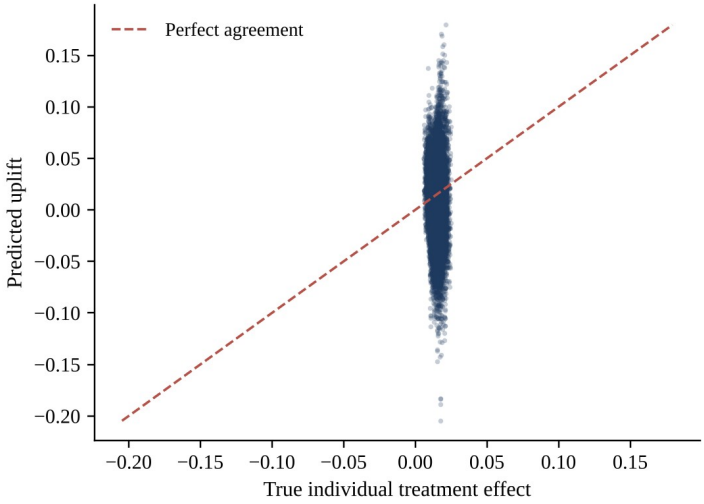



**Figure 4.** True individual treatment effect against predicted uplift.

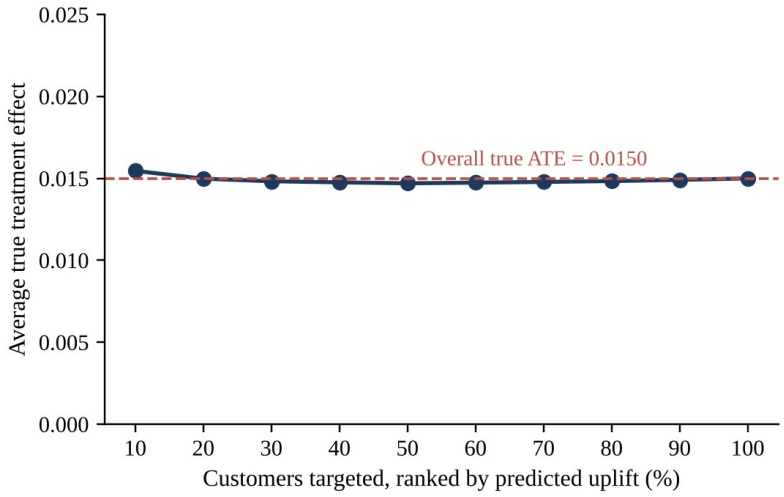



**Figure 5.** True average treatment effect by share of customers targeted.

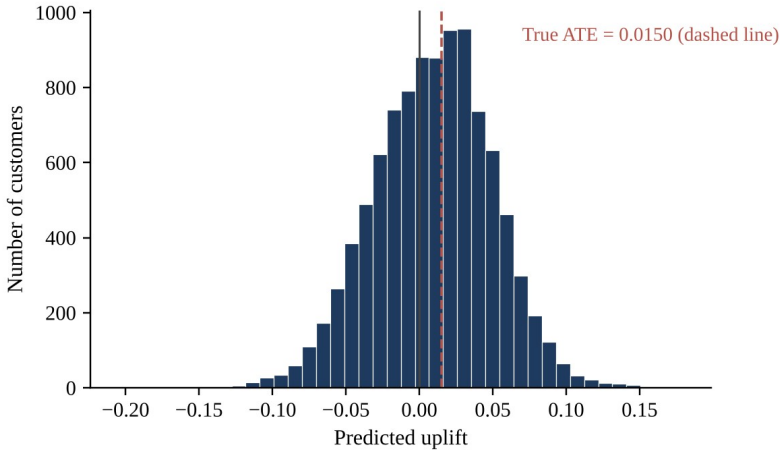



**Figure 6.** Distribution of predicted uplift.

## 13.4 Bootstrap and Placebo Figures

Figure 7 shows the bootstrap interval for the observed difference and the placebo interval. The first excludes zero, and the second includes it.

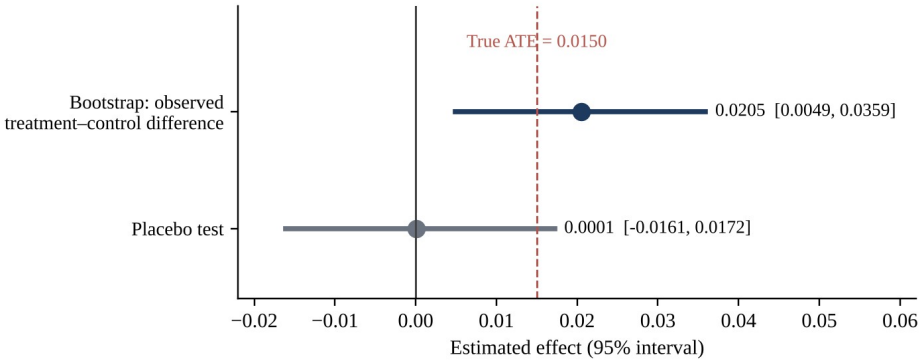



**Figure 7.** Bootstrap and placebo intervals for the estimated effect.

## 13.5 ROI and Revenue Figures

Figure 8 compares incremental revenue and expected incremental purchases for the three targeting strategies. The top 30% group leads on revenue but not on purchases.

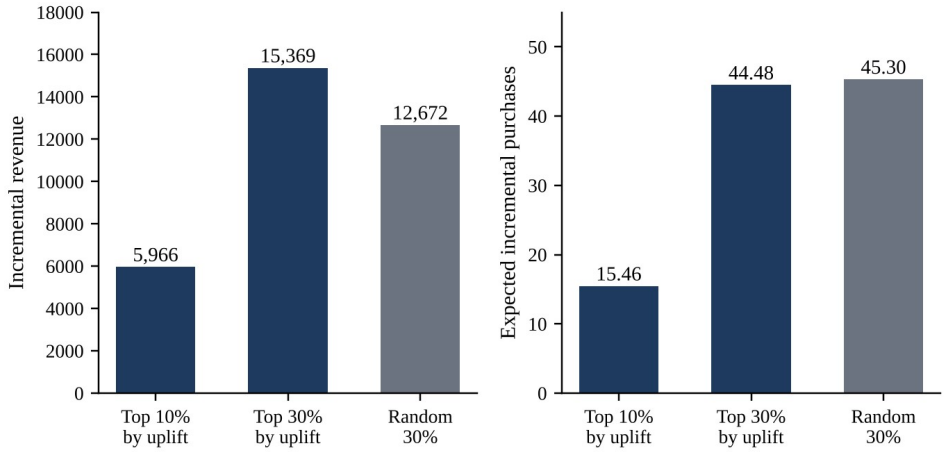



**Figure 8.** Incremental revenue and expected incremental purchases by targeting strategy.

# 14. DISCUSSION

## 14.1 Major Findings

The results show a clear progression. The naive estimate overstated the true effect of the discount by about 39%, and across ten repeated simulations it overstated the effect in nine cases, by about 73% on average. The causal forest produced a smaller error for the average effect, although it understated the effect. Estimates of individual effects, from the causal forest and from the two-model approach, were noisy and agreed weakly or not at all with the true effects, and the uplift ranking did not select customers with larger true effects.

## 14.2 Statistical Discussion

Bias, uncertainty, heterogeneity and robustness are all visible in the results. The naive estimator is biased by confounding. The bootstrap interval for the observed difference is wide, from 0.0049 to 0.0359, and the standard deviation of the naive estimate across seeds, 0.00813, is more than half the size of the true effect. The true effects vary little across customers, from 0.0059 to 0.0248, which makes individual differences hard to detect against the noise in purchase outcomes. The significant test result confirms a difference between the groups but, as Section 10.4 notes, does not by itself separate the causal effect from confounding.

## 14.3 Business Discussion

A model that predicts purchase tells a business which customers are likely to buy, but many of them would buy without a discount, and a discount given to them is a cost with no benefit. The incremental effect measures what the discount adds, and it is therefore the quantity that matters for decisions about whom to offer discounts. In this study, targeting the top 30% by predicted uplift produced more incremental revenue than random targeting, but slightly fewer incremental purchases. The revenue advantage came from the higher order values of the selected customers (Section 11.3), so the experiment does not show that the uplift model improved targeting.

## 14.4 Comparison of Methods

Table 13 compares the three methods used in the study.

**Table 13.** Strengths and weaknesses of the methods compared.

| Method | Strengths | Weaknesses |
|---|---|---|
| Naive estimation | Simple to compute and easy to explain | Biased under confounding; overstated the effect by 39.10% |

| Method | Strengths | Weaknesses |
|---|---|---|
| Causal forest | Adjusts for confounding; estimates both ATE and CATE; closer ATE (error 0.0048) | Understated the ATE; individual estimates noisy (correlation 0.1755) |
| Uplift modelling (two-model) | Intuitive; gives a ranking of customers for targeting | Weak ranking (correlation −0.0591); large RMSE (0.0413) |

# 15. LIMITATIONS

## 15.1 Synthetic Data

The data are simulated, and the treatment effect and confounding are specified by the simulation. They may not capture the behaviour of real customers.

## 15.2 Individual-Level Effect Estimation

Estimates of individual effects were noisy, and the correlation between predicted and true uplift was weak. Conclusions about the targeting of individual customers are therefore limited. In addition, the revenue comparison of Chapter 11 is driven by order value and should not be read as evidence of better targeting.

## 15.3 ROI Assumption

Because the cost of the discount is defined as 10% of incremental revenue, ROI is mechanically fixed at 900% for every strategy. A realistic cost model is needed for ROI to be informative.

## 15.4 Causal Interpretation

Causal conclusions from real observational data would require additional assumptions, such as that all important confounders are measured, together with further validation.

# 16. CONCLUSION

## 16.1 Summary of the Study

This study examined whether discounts cause additional purchases and how the effect can be estimated and used for targeting. In a synthetic dataset of 10,000 customers with a known true ATE of 0.0150, a naive comparison gave 0.0209, an overstatement of 39.10% caused by confounding by average order value. Across ten random seeds the naive estimate overstated the effect in nine cases, by about 73% on average. The causal forest estimated the ATE as 0.0102, with a smaller absolute error, but its individual-level estimates were noisy. The uplift model ranked customers weakly: the true average effect was almost the same in every targeted group. Statistical validation supported a difference between treated and control customers. Targeting the top 30% by predicted uplift generated more incremental revenue than random targeting (15,368.59 against 12,671.77) but slightly fewer incremental purchases (44.48 against 45.30), because the selected customers had higher order values.

## 16.2 Key Contribution

The key contribution of the study is its focus on estimating incremental effects and not only predicting purchases. It shows, in a setting where the truth is known, that simple comparisons can mislead and that causal methods are needed for decisions about interventions.

## 16.3 Final Interpretation

The results support a balanced conclusion. Causal machine learning improved the estimate of the average effect, but it did not reliably identify which individual customers respond most to a discount. The experiment does not show a business benefit from uplift targeting: the apparent revenue gain is explained by order value. Showing such a benefit would require better estimators and, ideally, randomised experiments.

# 17. FUTURE SCOPE

## 17.1 Real-World Dataset

The framework should be applied to a suitable real-world uplift or causal dataset, such as the Criteo uplift dataset (Diemert et al., 2018). Because true individual effects are unknown in real data, evaluation would use Qini and area under the uplift curve (AUUC) measures on held-out data.

## 17.2 Experimental Validation

Randomised A/B testing should be used where possible to validate the estimated effects and the targeting policy.

## 17.3 Advanced Uplift Methods

Stronger meta-learners, further causal forest variants and policy learning methods could improve the ranking of customers.

## 17.4 Improved ROI Modeling

A more realistic ROI model should use actual reward costs, margins and budget constraints.

## 17.5 Practical Deployment

Future work could adapt the framework to personalised digital-payment rewards or similar business applications.

# 18. REFERENCES

## 18.1 Academic References


Athey, S., Tibshirani, J., & Wager, S. (2019). Generalized random forests. *The Annals of Statistics, 47*(2), 1148–1178.

Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., & Robins, J. (2018). Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal, 21*(1), C1–C68.

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

Imbens, G. W., & Rubin, D. B. (2015). *Causal inference for statistics, social, and biomedical sciences: An introduction*. Cambridge University Press.

Künzel, S. R., Sekhon, J. S., Bickel, P. J., & Yu, B. (2019). Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the National Academy of Sciences, 116*(10), 4156–4165.

Pearl, J. (2009). *Causality: Models, reasoning, and inference* (2nd ed.). Cambridge University Press.

Rosenbaum, P. R., & Rubin, D. B. (1983). The central role of the propensity score in observational studies for causal effects. *Biometrika, 70*(1), 41–55.

Rubin, D. B. (1974). Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology, 66*(5), 688–701.

Wager, S., & Athey, S. (2018). Estimation and inference of heterogeneous treatment effects using random forests. *Journal of the American Statistical Association, 113*(523), 1228–1242.


## 18.2 Software Documentation


Battocchi, K., Dillon, E., Hei, M., Lewis, G., Oka, P., Oprescu, M., & Syrgkanis, V. (2019). *EconML: A Python package for ML-based heterogeneous treatment effects estimation* [Computer software]. Microsoft Research.

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature, 585*, 357–362.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.


## 18.3 Dataset and Method References


Diemert, E., Betlei, A., Renaudin, C., & Amini, M.-R. (2018). A large scale benchmark for uplift modeling. *Proceedings of the AdKDD and TargetAd Workshop, KDD 2018*.

# 19. APPENDIX

## Appendix A: Python Code

The code below was used for Steps 1 to 6 in the Colab notebook, and for the robustness analysis of Step 7, with explanatory comments and blank lines removed.

**Step 1: Synthetic data generation**

```
import os
import pandas as pd
import numpy as np
np.random.seed(42)
n_customers = 10_000
os.makedirs("data", exist_ok=True)
data = pd.DataFrame({
    "previous_purchases":
        np.random.randint(0, 50, n_customers),
    "avg_order_value":
        np.random.uniform(10, 500, n_customers),
    "days_since_last_purchase":
        np.random.randint(1, 365, n_customers),
    "email_clicked":
        np.random.binomial(1, 0.35, n_customers)
})
prob_treatment = (
    0.20
    + 0.50 * (
        data["avg_order_value"] / 500
    )
)
prob_treatment = np.clip(
    prob_treatment,
    0.05,
    0.90
)
data["treatment"] = (
    np.random.random(n_customers)
    < prob_treatment
).astype(int)
baseline_prob = (
    0.10
    + 0.002 * data["previous_purchases"]
    + 0.00015 * data["avg_order_value"]
    + 0.05 * data["email_clicked"]
)
baseline_prob = np.clip(
    baseline_prob,
    0.01,
    0.80
)
raw_effect = (
    0.005
    + 0.00015 * data["previous_purchases"]
    + 0.000015 * data["avg_order_value"]
    + 0.002 * data["email_clicked"]
)
target_ate = 0.015
scaling_factor = (
    target_ate / raw_effect.mean()
)
true_individual_effect = (
    raw_effect * scaling_factor
)
data["true_individual_effect"] = (
    true_individual_effect
)
prob_without_discount = baseline_prob
prob_with_discount = (
    baseline_prob
```

```
    + true_individual_effect
)
prob_with_discount = np.clip(
    prob_with_discount,
    0.01,
    0.95
)
observed_probability = np.where(
    data["treatment"] == 1,
    prob_with_discount,
    prob_without_discount
)
data["outcome"] = (
    np.random.random(n_customers)
    < observed_probability
).astype(int)
file_path = "data/synthetic_data.csv"
data.to_csv(
    file_path,
    index=False
)
true_ate = (
    data["true_individual_effect"].mean()
)
print("=" * 65)
print("STEP 1: SYNTHETIC CAUSAL DATA GENERATED")
print("=" * 65)
print(
    f"Number of customers: "
    f"{n_customers}"
)
print(
    f"Treatment rate: "
    f"{data['treatment'].mean():.2%}"
)
print(
    f"Overall purchase rate: "
    f"{data['outcome'].mean():.2%}"
)
print("-" * 65)
print(
    f"Purchase rate - No discount: "
    f"{data.loc[data['treatment'] == 0, 'outcome'].mean():.2%}"
)
print(
    f"Purchase rate - Discount: "
    f"{data.loc[data['treatment'] == 1, 'outcome'].mean():.2%}"
)
print("-" * 65)
print(
    f"TRUE ATE: "
    f"{true_ate:.4f}"
)
print(
    f"TRUE ATE in percentage points: "
    f"{true_ate * 100:.2f}%"
)
print("-" * 65)
print(
    f"Minimum true individual effect: "
    f"{data['true_individual_effect'].min():.4f}"
)
print(
    f"Maximum true individual effect: "
    f"{data['true_individual_effect'].max():.4f}"
)
print(
    f"Median true individual effect: "
    f"{data['true_individual_effect'].median():.4f}"
)
print("-" * 65)
treated_aov = data.loc[
    data["treatment"] == 1,
```

```
    "avg_order_value"
].mean()
control_aov = data.loc[
    data["treatment"] == 0,
    "avg_order_value"
].mean()
print("CONFOUNDING CHECK")
print(
    f"Average order value - Treated: "
    f"{treated_aov:.2f}"
)
print(
    f"Average order value - Control: "
    f"{control_aov:.2f}"
)
print("=" * 65)
print(
    f"Data saved to: {file_path}"
)
```

### Step 2: Naive estimation

```
import pandas as pd
import numpy as np
data = pd.read_csv("data/synthetic_data.csv")
treated = data[data["treatment"] == 1]
control = data[data["treatment"] == 0]
treated_mean = treated["outcome"].mean()
control_mean = control["outcome"].mean()
naive_ate = (
    treated_mean
    - control_mean
)
true_ate = (
    data["true_individual_effect"].mean()
)
bias = naive_ate - true_ate
absolute_error = abs(bias)
relative_error = (
    absolute_error / abs(true_ate)
) * 100
treated_aov = treated["avg_order_value"].mean()
control_aov = control["avg_order_value"].mean()
print("=" * 65)
print("STEP 2: NAIVE ESTIMATION")
print("=" * 65)
print(
    f"Treated customers: "
    f"{len(treated)}"
)
print(
    f"Control customers: "
    f"{len(control)}"
)
print("-" * 65)
print(
    f"Treated purchase rate: "
    f"{treated_mean:.4f}"
)
print(
    f"Control purchase rate: "
    f"{control_mean:.4f}"
)
print("-" * 65)
print(
    f"Naive ATE: "
    f"{naive_ate:.4f}"
)
print(
    f"True ATE: "
    f"{true_ate:.4f}"
)
print(
```

```
    f"Bias: "
    f"{bias:.4f}"
)
print(
    f"Absolute error: "
    f"{absolute_error:.4f}"
)
print(
    f"Relative error: "
    f"{relative_error:.2f}%"
)
print("-" * 65)
print("CONFOUNDING CHECK")
print(
    f"Average order value - Treated: "
    f"{treated_aov:.2f}"
)
print(
    f"Average order value - Control: "
    f"{control_aov:.2f}"
)
print("=" * 65)
print("\nINTERPRETATION")
print("=" * 65)
print(
    "The naive estimator compares the observed purchase "
    "rates of discounted and non-discounted customers."
)
print(
    "Because high-value customers are more likely to receive "
    "discounts, treatment assignment is confounded."
)
print(
    f"The naive estimate is {naive_ate:.4f}, "
    f"while the true ATE is {true_ate:.4f}."
)
if naive_ate > true_ate:
    print(
        "Therefore, the naive method overestimates the "
        "average causal effect of the discount."
    )
else:
    print(
        "Therefore, the naive method underestimates the "
        "average causal effect of the discount."
    )
```

### Step 3: Causal forest

```
import pandas as pd
import numpy as np
from econml.dml import CausalForestDML
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
data = pd.read_csv("data/synthetic_data.csv")
feature_columns = [
    "previous_purchases",
    "avg_order_value",
    "days_since_last_purchase",
    "email_clicked"
]
X = data[feature_columns]
T = data["treatment"].astype(float)
Y = data["outcome"].astype(float)
true_cate = data["true_individual_effect"].values
true_ate = true_cate.mean()
print("Training Causal Forest...")
cf = CausalForestDML(
    model_y=RandomForestRegressor(
        n_estimators=200,
        min_samples_leaf=10,
        random_state=42
    ),
```

```
    model_t=RandomForestRegressor(
        n_estimators=200,
        min_samples_leaf=10,
        random_state=42
    ),
    n_estimators=300,
    min_samples_leaf=10,
    max_depth=5,
    random_state=42
)
cf.fit(
    Y,
    T,
    X=X
)
estimated_cate = cf.effect(X)
estimated_ate = estimated_cate.mean()
ate_error = (
    estimated_ate - true_ate
)
ate_absolute_error = abs(ate_error)
ate_relative_error = (
    ate_absolute_error /
    abs(true_ate)
) * 100
naive_ate = (
    data.loc[
        data["treatment"] == 1,
        "outcome"
    ].mean()
    -
    data.loc[
        data["treatment"] == 0,
        "outcome"
    ].mean()
)
naive_absolute_error = abs(
    naive_ate - true_ate
)
naive_relative_error = (
    naive_absolute_error /
    abs(true_ate)
) * 100
cate_rmse = np.sqrt(
    mean_squared_error(
        true_cate,
        estimated_cate
    )
)
cate_correlation = np.corrcoef(
    true_cate,
    estimated_cate
)[0, 1]
data["estimated_cate"] = estimated_cate
data["cate_error"] = (
    data["estimated_cate"]
    -
    data["true_individual_effect"]
)
print("=" * 65)
print("STEP 3: CAUSAL FOREST RESULTS")
print("=" * 65)
print(
    f"True ATE:              "
    f"{true_ate:.4f}"
)
print(
    f"Naive ATE:              "
    f"{naive_ate:.4f}"
)
print(
    f"Causal Forest ATE:     "
    f"{estimated_ate:.4f}"
```

```
)
print("-" * 65)
print(
    f"Naive absolute error:   "
    f"{naive_absolute_error:.4f}"
)
print(
    f"Causal Forest absolute error: "
    f"{ate_absolute_error:.4f}"
)
print("-" * 65)
print(
    f"Naive relative error:   "
    f"{naive_relative_error:.2f}%"
)
print(
    f"Causal Forest relative error: "
    f"{ate_relative_error:.2f}%"
)
print("=" * 65)
print("\nHETEROGENEOUS TREATMENT EFFECTS")
print("=" * 65)
print(
    f"True minimum effect: "
    f"{true_cate.min():.4f}"
)
print(
    f"True maximum effect: "
    f"{true_cate.max():.4f}"
)
print(
    f"Estimated minimum effect: "
    f"{estimated_cate.min():.4f}"
)
print(
    f"Estimated maximum effect: "
    f"{estimated_cate.max():.4f}"
)
print("-" * 65)
print(
    f"CATE RMSE: "
    f"{cate_rmse:.4f}"
)
print(
    f"True vs estimated CATE correlation: "
    f"{cate_correlation:.4f}"
)
print("\nTOP 10 CUSTOMERS BY ESTIMATED TREATMENT EFFECT")
print("=" * 65)
top_customers = (
    data
    .sort_values(
        "estimated_cate",
        ascending=False
    )
    .head(10)
)
print(
    top_customers[
        feature_columns
        +
        [
            "true_individual_effect",
            "estimated_cate"
        ]
    ].to_string(index=False)
)
print("\nBUSINESS INTERPRETATION")
print("=" * 65)
if ate_absolute_error < naive_absolute_error:
    print(
        "Causal Forest produced an ATE closer to the "
        "true ATE than the naive estimator."
```

```
    )
else:
    print(
        "Causal Forest did not produce an ATE closer "
        "to the true ATE than the naive estimator."
    )
if cate_correlation > 0:
    print(
        "The positive CATE correlation indicates that "
        "the model is capturing some of the differences "
        "in treatment response across customers."
    )
else:
    print(
        "The CATE correlation is not positive, indicating "
        "that the model is not reliably identifying "
        "individual treatment-effect differences."
    )
print(
    "These estimated individual effects can be used "
    "in Step 4 to target customers who are predicted "
    "to benefit most from discounts."
)
```

### Step 4: Cross-fitted two-model uplift modelling

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import mean_squared_error
os.makedirs("results/plots", exist_ok=True)
data = pd.read_csv(
    "data/synthetic_data.csv"
)
feature_columns = [
    "previous_purchases",
    "avg_order_value",
    "days_since_last_purchase",
    "email_clicked"
]
X = data[feature_columns]
T = data["treatment"].values
Y = data["outcome"].values
true_effect = data[
    "true_individual_effect"
].values
predicted_purchase_if_treated = np.zeros(
    len(data)
)
predicted_purchase_if_control = np.zeros(
    len(data)
)
skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
print("=" * 65)
print("STEP 4: CROSS-FITTED TWO-MODEL UPLIFT MODEL")
print("=" * 65)
print("\nTraining 5-fold uplift models...")
for fold, (train_index, test_index) in enumerate(
    skf.split(X, T),
    start=1
):
    print(f"Processing fold {fold}/5...")
    X_train = X.iloc[train_index]
    T_train = T[train_index]
    Y_train = Y[train_index]
    X_test = X.iloc[test_index]
```

```
    treatment_mask = (
        T_train == 1
    )
    X_treatment = X_train[
        treatment_mask
    ]
    Y_treatment = Y_train[
        treatment_mask
    ]
    control_mask = (
        T_train == 0
    )
    X_control = X_train[
        control_mask
    ]
    Y_control = Y_train[
        control_mask
    ]
    treatment_model = RandomForestClassifier(
        n_estimators=300,
        max_depth=5,
        min_samples_leaf=30,
        random_state=42 + fold,
        n_jobs=-1
    )
    treatment_model.fit(
        X_treatment,
        Y_treatment
    )
    control_model = RandomForestClassifier(
        n_estimators=300,
        max_depth=5,
        min_samples_leaf=30,
        random_state=100 + fold,
        n_jobs=-1
    )
    control_model.fit(
        X_control,
        Y_control
    )
    predicted_purchase_if_treated[
        test_index
    ] = treatment_model.predict_proba(
        X_test
    )[:, 1]
    predicted_purchase_if_control[
        test_index
    ] = control_model.predict_proba(
        X_test
    )[:, 1]
predicted_uplift = (
    predicted_purchase_if_treated
    -
    predicted_purchase_if_control
)
data[
    "purchase_if_discount"
] = predicted_purchase_if_treated
data[
    "purchase_if_no_discount"
] = predicted_purchase_if_control
data[
    "predicted_uplift"
] = predicted_uplift
uplift_rmse = np.sqrt(
    mean_squared_error(
        true_effect,
        predicted_uplift
    )
)
uplift_correlation = np.corrcoef(
    true_effect,
    predicted_uplift
```

```
)[0, 1]
print("\n" + "=" * 65)
print("UPLIFT MODEL RESULTS")
print("=" * 65)
print(
    f"Mean predicted uplift: "
    f"{predicted_uplift.mean():.4f}"
)
print(
    f"Mean true treatment effect: "
    f"{true_effect.mean():.4f}"
)
print("-" * 65)
print(
    f"Uplift RMSE: "
    f"{uplift_rmse:.4f}"
)
print(
    f"True vs predicted uplift correlation: "
    f"{uplift_correlation:.4f}"
)
data = data.sort_values(
    "predicted_uplift",
    ascending=False
).reset_index(
    drop=True
)
data[
    "target_rank"
] = np.arange(
    1,
    len(data) + 1
)
percentages = [
    10, 20, 30, 40, 50,
    60, 70, 80, 90, 100
]
targeting_results = []
for percentage in percentages:
    n_target = int(
        len(data) * percentage / 100
    )
    selected = data.iloc[
        :n_target
    ]
    mean_true_effect = (
        selected[
            "true_individual_effect"
        ].mean()
    )
    mean_predicted_uplift = (
        selected[
            "predicted_uplift"
        ].mean()
    )
    targeting_results.append({
        "percentage_targeted":
            percentage,
        "customers_targeted":
            n_target,
        "mean_predicted_uplift":
            mean_predicted_uplift,
        "mean_true_effect":
            mean_true_effect
    })
targeting_results = pd.DataFrame(
    targeting_results
)
print("\n" + "=" * 65)
print("TARGETING PERFORMANCE")
print("=" * 65)
print(
    targeting_results.to_string(
```

```
        index=False
    )
)
print("\n" + "=" * 65)
print("TOP 20 CUSTOMERS FOR DISCOUNT TARGETING")
print("=" * 65)
print(
    data[
        [
            "target_rank",
            "previous_purchases",
            "avg_order_value",
            "days_since_last_purchase",
            "email_clicked",
            "true_individual_effect",
            "predicted_uplift"
        ]
    ].head(20).to_string(
        index=False
    )
)
top_10 = data.iloc[
    :int(0.10 * len(data))
]
top_30 = data.iloc[
    :int(0.30 * len(data))
]
remaining = data.iloc[
    int(0.30 * len(data)) :
]
print("\n" + "=" * 65)
print("BUSINESS TARGETING COMPARISON")
print("=" * 65)
print(
    f"Top 10% true average effect: "
    f"{top_10['true_individual_effect'].mean():.4f}"
)
print(
    f"Top 30% true average effect: "
    f"{top_30['true_individual_effect'].mean():.4f}"
)
print(
    f"Remaining 70% true average effect: "
    f"{remaining['true_individual_effect'].mean():.4f}"
)
overall_true_ate = (
    data[
        "true_individual_effect"
    ].mean()
)
top_10_true_effect = (
    top_10[
        "true_individual_effect"
    ].mean()
)
targeting_lift = (
    top_10_true_effect
    -
    overall_true_ate
)
print(
    f"\nTop 10% targeting lift over "
    f"overall ATE: {targeting_lift:.4f}"
)
plt.figure(
    figsize=(8, 6)
)
plt.scatter(
    true_effect,
    predicted_uplift,
    alpha=0.30
)
minimum = min(
```

```
    true_effect.min(),
    predicted_uplift.min()
)
maximum = max(
    true_effect.max(),
    predicted_uplift.max()
)
plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    linestyle="--"
)
plt.xlabel(
    "True Individual Treatment Effect"
)
plt.ylabel(
    "Predicted Uplift"
)
plt.title(
    "True vs Predicted Individual Treatment Effect"
)
plt.tight_layout()
plt.savefig(
    "results/plots/04_true_vs_predicted_uplift.png",
    dpi=300
)
plt.close()
plt.figure(
    figsize=(8, 6)
)
plt.plot(
    targeting_results[
        "percentage_targeted"
    ],
    targeting_results[
        "mean_true_effect"
    ],
    marker="o"
)
plt.axhline(
    y=overall_true_ate,
    linestyle="--",
    label="Overall True ATE"
)
plt.xlabel(
    "Percentage of Customers Targeted"
)
plt.ylabel(
    "Average True Treatment Effect"
)
plt.title(
    "Uplift Targeting Curve"
)
plt.legend()
plt.tight_layout()
plt.savefig(
    "results/plots/05_uplift_targeting_curve.png",
    dpi=300
)
plt.close()
plt.figure(
    figsize=(8, 6)
)
plt.hist(
    predicted_uplift,
    bins=40,
    edgecolor="black"
)
plt.axvline(
    x=0,
    linestyle="--"
)
plt.xlabel(
```

```
    "Predicted Uplift"
)
plt.ylabel(
    "Number of Customers"
)
plt.title(
    "Distribution of Predicted Customer Uplift"
)
plt.tight_layout()
plt.savefig(
    "results/plots/06_uplift_distribution.png",
    dpi=300
)
plt.close()
data[
    feature_columns
    + [
        "treatment",
        "outcome",
        "true_individual_effect",
        "purchase_if_discount",
        "purchase_if_no_discount",
        "predicted_uplift",
        "target_rank"
    ]
].to_csv(
    "results/uplift_targeting_results.csv",
    index=False
)
targeting_results.to_csv(
    "results/targeting_performance.csv",
    index=False
)
print("\n" + "=" * 65)
print("STEP 4 COMPLETED")
print("=" * 65)
if (
    top_10_true_effect
    > overall_true_ate
):
    print(
        "The top 10% targeted customers have a "
        "higher average true treatment effect than "
        "the overall population."
    )
else:
    print(
        "The top 10% targeted customers do not have "
        "a higher average true treatment effect than "
        "the overall population."
    )
print(
    "\nTargeting results saved to:"
)
print(
    "results/uplift_targeting_results.csv"
)
print(
    "results/targeting_performance.csv"
)
print("\nPlots saved to:")
print(
    "results/plots/04_true_vs_predicted_uplift.png"
)
print(
    "results/plots/05_uplift_targeting_curve.png"
)
print(
    "results/plots/06_uplift_distribution.png"
)
print("=" * 65)
```

### Step 5: Statistical validation

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.linear_model import LinearRegression
os.makedirs(
    "results/plots",
    exist_ok=True
)
data = pd.read_csv(
    "data/synthetic_data.csv"
)
feature_columns = [
    "previous_purchases",
    "avg_order_value",
    "days_since_last_purchase",
    "email_clicked"
]
X = data[feature_columns]
T = data["treatment"].values
Y = data["outcome"].values
treated_mean = Y[T == 1].mean()
control_mean = Y[T == 0].mean()
naive_ate = (
    treated_mean -
    control_mean
)
true_ate = data[
    "true_individual_effect"
].mean()
print("=" * 65)
print("STEP 5: STATISTICAL VALIDATION")
print("=" * 65)
print(
    f"Naive ATE: {naive_ate:.4f}"
)
print(
    f"True ATE:  {true_ate:.4f}"
)
print("\n" + "=" * 65)
print("PART A: BOOTSTRAP CONFIDENCE INTERVAL")
print("=" * 65)
n_bootstrap = 500
bootstrap_ates = []
rng = np.random.default_rng(42)
for i in range(n_bootstrap):
    sample_indices = rng.choice(
        len(data),
        size=len(data),
        replace=True
    )
    sample = data.iloc[
        sample_indices
    ]
    if (
        sample["treatment"].nunique()
        < 2
    ):
        continue
    treated = sample[
        sample["treatment"] == 1
    ]["outcome"].mean()
    control = sample[
        sample["treatment"] == 0
    ]["outcome"].mean()
    bootstrap_ate = (
        treated -
        control
    )
    bootstrap_ates.append(
```

```
        bootstrap_ate
    )
bootstrap_ates = np.array(
    bootstrap_ates
)
lower_ci = np.percentile(
    bootstrap_ates,
    2.5
)
upper_ci = np.percentile(
    bootstrap_ates,
    97.5
)
bootstrap_mean = (
    bootstrap_ates.mean()
)
print(
    f"Bootstrap replications: "
    f"{len(bootstrap_ates)}"
)
print(
    f"Bootstrap mean ATE: "
    f"{bootstrap_mean:.4f}"
)
print(
    f"95% Bootstrap CI: "
    f"[{lower_ci:.4f}, {upper_ci:.4f}]"
)
if (
    lower_ci <= 0
    and upper_ci >= 0
):
    print(
        "The confidence interval contains zero."
    )
else:
    print(
        "The confidence interval does not contain zero."
    )
plt.figure(
    figsize=(9, 6)
)
plt.hist(
    bootstrap_ates,
    bins=40,
    edgecolor="black"
)
plt.axvline(
    lower_ci,
    linestyle="--",
    linewidth=2,
    label=f"2.5% = {lower_ci:.4f}"
)
plt.axvline(
    upper_ci,
    linestyle="--",
    linewidth=2,
    label=f"97.5% = {upper_ci:.4f}"
)
plt.axvline(
    true_ate,
    linestyle="-",
    linewidth=2,
    label=f"True ATE = {true_ate:.4f}"
)
plt.xlabel(
    "Bootstrap ATE"
)
plt.ylabel(
    "Frequency"
)
plt.title(
    "Bootstrap Distribution of ATE"
```

```
)
plt.legend()
plt.tight_layout()
plt.savefig(
    "results/plots/07_bootstrap_ate.png",
    dpi=300
)
plt.close()
print("\n" + "=" * 65)
print("PART B: HYPOTHESIS TEST")
print("=" * 65)
treated_outcomes = data[
    data["treatment"] == 1
]["outcome"]
control_outcomes = data[
    data["treatment"] == 0
]["outcome"]
t_statistic, p_value = stats.ttest_ind(
    treated_outcomes,
    control_outcomes,
    equal_var=False
)
print(
    f"T-statistic: "
    f"{t_statistic:.4f}"
)
print(
    f"P-value: "
    f"{p_value:.6f}"
)
alpha = 0.05
if p_value < alpha:
    print(
        "Decision: Reject H0 at the 5% significance level."
    )
    print(
        "There is statistically significant evidence "
        "of a difference in purchase rates."
    )
else:
    print(
        "Decision: Fail to reject H0 at the 5% significance level."
    )
    print(
        "There is insufficient evidence of a difference "
        "in purchase rates."
    )
print("\n" + "=" * 65)
print("PART C: PLACEBO TEST")
print("=" * 65)
n_placebo = 500
placebo_ates = []
for i in range(n_placebo):
    placebo_T = rng.permutation(T)
    placebo_treated = Y[
        placebo_T == 1
    ].mean()
    placebo_control = Y[
        placebo_T == 0
    ].mean()
    placebo_ate = (
        placebo_treated -
        placebo_control
    )
    placebo_ates.append(
        placebo_ate
    )
placebo_ates = np.array(
    placebo_ates
)
placebo_mean = (
    placebo_ates.mean()
)
```

```
placebo_lower = np.percentile(
    placebo_ates,
    2.5
)
placebo_upper = np.percentile(
    placebo_ates,
    97.5
)
print(
    f"Mean placebo ATE: "
    f"{placebo_mean:.4f}"
)
print(
    f"95% placebo interval: "
    f"[{placebo_lower:.4f}, "
    f"{placebo_upper:.4f}]"
)
plt.figure(
    figsize=(9, 6)
)
plt.hist(
    placebo_ates,
    bins=40,
    edgecolor="black"
)
plt.axvline(
    0,
    linestyle="--",
    linewidth=2,
    label="Zero effect"
)
plt.axvline(
    naive_ate,
    linestyle="-",
    linewidth=2,
    label=f"Observed ATE = {naive_ate:.4f}"
)
plt.xlabel(
    "Placebo ATE"
)
plt.ylabel(
    "Frequency"
)
plt.title(
    "Placebo Treatment Effect Distribution"
)
plt.legend()
plt.tight_layout()
plt.savefig(
    "results/plots/08_placebo_test.png",
    dpi=300
)
plt.close()
print("\n" + "=" * 65)
print("STEP 5 SUMMARY")
print("=" * 65)
print(
    f"True ATE:              {true_ate:.4f}"
)
print(
    f"Naive ATE:             {naive_ate:.4f}"
)
print(
    f"Bootstrap mean ATE:    {bootstrap_mean:.4f}"
)
print(
    f"95% Bootstrap CI:      "
    f"[{lower_ci:.4f}, {upper_ci:.4f}]"
)
print(
    f"Hypothesis test p-value: "
    f"{p_value:.6f}"
)
```

```
print(
    f"Mean placebo ATE:      "
    f"{placebo_mean:.4f}"
)
print(
    f"95% placebo interval:  "
    f"[{placebo_lower:.4f}, "
    f"{placebo_upper:.4f}]"
)
print("=" * 65)
print(
    "\nPlots saved to:"
)
print(
    "results/plots/07_bootstrap_ate.png"
)
print(
    "results/plots/08_placebo_test.png"
)
print("=" * 65)
```

### Step 6: ROI and targeting strategy

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
os.makedirs(
    "results/plots",
    exist_ok=True
)
data = pd.read_csv(
    "results/uplift_targeting_results.csv"
)
DISCOUNT_RATE = 0.10
RANDOM_SEED = 42
np.random.seed(
    RANDOM_SEED
)
print("=" * 65)
print("STEP 6: ROI & TARGETING STRATEGY")
print("=" * 65)
print("\nBUSINESS ASSUMPTIONS")
print("-" * 65)
print(
    f"Discount rate: "
    f"{DISCOUNT_RATE:.0%}"
)
print(
    "Revenue per incremental purchase = "
    "Average Order Value"
)
print(
    "Discount cost is calculated only on "
    "incremental purchases."
)
def calculate_roi(customers):
    incremental_purchases = (
        customers[
            "true_individual_effect"
        ].sum()
    )
    incremental_revenue = (
        customers[
            "true_individual_effect"
        ]
        *
        customers[
            "avg_order_value"
        ]
    ).sum()
    discount_cost = (
        customers[
```

```
            "true_individual_effect"
        ]
        *
        customers[
            "avg_order_value"
        ]
        *
        DISCOUNT_RATE
    ).sum()
    net_incremental_value = (
        incremental_revenue
        -
        discount_cost
    )
    if discount_cost > 0:
        roi = (
            net_incremental_value
            /
            discount_cost
        ) * 100
    else:
        roi = np.nan
    return {
        "customers":
            len(customers),
        "incremental_purchases":
            incremental_purchases,
        "incremental_revenue":
            incremental_revenue,
        "discount_cost":
            discount_cost,
        "net_incremental_value":
            net_incremental_value,
        "ROI_percent":
            roi
    }
data = data.sort_values(
    "predicted_uplift",
    ascending=False
).reset_index(
    drop=True
)
n_10 = int(
    0.10 * len(data)
)
top_10 = data.iloc[
    :n_10
]
result_10 = calculate_roi(
    top_10
)
n_30 = int(
    0.30 * len(data)
)
top_30 = data.iloc[
    :n_30
]
result_30 = calculate_roi(
    top_30
)
random_30 = data.sample(
    n=n_30,
    random_state=RANDOM_SEED
)
result_random = calculate_roi(
    random_30
)
roi_results = pd.DataFrame([
    {
        "strategy":
            "Top 10% Uplift Targeting",
        **result_10
    },
```

```
    {
        "strategy":
            "Top 30% Uplift Targeting",
        **result_30
    },
    {
        "strategy":
            "Random 30% Targeting",
        **result_random
    }
])
print("\n" + "=" * 65)
print("ROI COMPARISON")
print("=" * 65)
print(
    roi_results.to_string(
        index=False,
        formatters={
            "incremental_purchases":
                "{:.2f}".format,
            "incremental_revenue":
                "{:.2f}".format,
            "discount_cost":
                "{:.2f}".format,
            "net_incremental_value":
                "{:.2f}".format,
            "ROI_percent":
                "{:.2f}%".format
        }
    )
)
top30_roi = (
    result_30[
        "ROI_percent"
    ]
)
random30_roi = (
    result_random[
        "ROI_percent"
    ]
)
roi_gain = (
    top30_roi -
    random30_roi
)
print("\n" + "=" * 65)
print("TARGETING STRATEGY COMPARISON")
print("=" * 65)
print(
    f"Top 30% uplift ROI: "
    f"{top30_roi:.2f}%"
)
print(
    f"Random 30% ROI: "
    f"{random30_roi:.2f}%"
)
print(
    f"ROI improvement: "
    f"{roi_gain:.2f} percentage points"
)
if random30_roi != 0:
    relative_improvement = (
        (
            top30_roi -
            random30_roi
        )
        /
        abs(random30_roi)
    ) * 100
else:
    relative_improvement = np.nan
print(
    f"Relative ROI improvement: "
```

```
    f"{relative_improvement:.2f}%"
)
print("\n" + "=" * 65)
print("BUSINESS INTERPRETATION")
print("=" * 65)
if top30_roi > random30_roi:
    print(
        "The uplift-based Top 30% strategy "
        "outperforms random targeting."
    )
    print(
        "This indicates that the uplift model "
        "provides some value for discount targeting."
    )
else:
    print(
        "The uplift-based Top 30% strategy "
        "does not outperform random targeting."
    )
    print(
        "The current model does not provide enough "
        "individual-level targeting value."
    )
top30_incremental_revenue = (
    result_30[
        "incremental_revenue"
    ]
)
random30_incremental_revenue = (
    result_random[
        "incremental_revenue"
    ]
)
revenue_gain = (
    top30_incremental_revenue
    -
    random30_incremental_revenue
)
print("\n" + "=" * 65)
print("INCREMENTAL REVENUE COMPARISON")
print("=" * 65)
print(
    f"Top 30% incremental revenue: "
    f"{top30_incremental_revenue:.2f}"
)
print(
    f"Random 30% incremental revenue: "
    f"{random30_incremental_revenue:.2f}"
)
print(
    f"Revenue gain from uplift targeting: "
    f"{revenue_gain:.2f}"
)
plt.figure(
    figsize=(9, 6)
)
strategies = roi_results[
    "strategy"
]
roi_values = roi_results[
    "ROI_percent"
]
plt.bar(
    strategies,
    roi_values,
    edgecolor="black"
)
plt.ylabel(
    "ROI (%)"
)
plt.xlabel(
    "Targeting Strategy"
)
```

```
plt.title(
    "ROI Comparison of Discount Targeting Strategies"
)
plt.xticks(
    rotation=15
)
plt.axhline(
    y=0,
    linestyle="--"
)
plt.tight_layout()
plt.savefig(
    "results/plots/09_roi_comparison.png",
    dpi=300
)
plt.close()
plt.figure(
    figsize=(9, 6)
)
revenue_values = [
    result_10[
        "incremental_revenue"
    ],
    result_30[
        "incremental_revenue"
    ],
    result_random[
        "incremental_revenue"
    ]
]
plt.bar(
    strategies,
    revenue_values,
    edgecolor="black"
)
plt.ylabel(
    "Expected Incremental Revenue"
)
plt.xlabel(
    "Targeting Strategy"
)
plt.title(
    "Expected Incremental Revenue by Targeting Strategy"
)
plt.xticks(
    rotation=15
)
plt.tight_layout()
plt.savefig(
    "results/plots/10_incremental_revenue.png",
    dpi=300
)
plt.close()
roi_results.to_csv(
    "results/roi_targeting_results.csv",
    index=False
)
print("\n" + "=" * 65)
print("STEP 6 COMPLETED")
print("=" * 65)
print(
    "ROI results saved to:"
)
print(
    "results/roi_targeting_results.csv"
)
print("\nPlots saved to:")
print(
    "results/plots/09_roi_comparison.png"
)
print(
    "results/plots/10_incremental_revenue.png"
)
```

```
print("=" * 65)
```

**Step 7: Robustness analysis**

```
import numpy as np
import pandas as pd

def naive_ate(seed, n=10_000):
    np.random.seed(seed)
    pp  = np.random.randint(0, 50, n)
    aov = np.random.uniform(10, 500, n)
    dsl = np.random.randint(1, 365, n)
    em  = np.random.binomial(1, 0.35, n)
    pt  = np.clip(0.20 + 0.50 * aov / 500, 0.05, 0.90)
    T   = (np.random.random(n) < pt).astype(int)
    b   = np.clip(0.10 + 0.002*pp + 0.00015*aov + 0.05*em, 0.01, 0.80)
    raw = 0.005 + 0.00015*pp + 0.000015*aov + 0.002*em
    eff = raw * 0.015 / raw.mean()
    p1  = np.clip(b + eff, 0.01, 0.95)
    Y   = (np.random.random(n) < np.where(T == 1, p1, b)).astype(int)
    return Y[T == 1].mean() - Y[T == 0].mean(), eff.mean()

seeds = [42, 100, 200, 300, 400, 500, 600, 700, 800, 900]
res = pd.DataFrame([naive_ate(s) for s in seeds], columns=["naive", "true"], index=seeds)
res["bias"] = res.naive - res.true
print(res.round(4))
print("mean naive", res.naive.mean(), "SD", res.naive.std(),
      "mean bias", res.bias.mean(), "MAE", res.bias.abs().mean(),
      "max abs error", res.bias.abs().max())
```

## Appendix B: Detailed Output

Table 14 lists the ten customers with the highest causal forest estimates of the treatment effect, together with their true effects.

**Table 14.** Ten highest causal forest estimates of the individual treatment effect.

| Previous purchases | Average order value | Days since last purchase | Email clicked | True effect | Estimated effect |
|---|---|---|---|---|---|
| 41 | 496.7 | 29 | 0 | 0.0211 | 0.1000 |
| 41 | 496.1 | 31 | 0 | 0.0211 | 0.0997 |
| 44 | 492.8 | 37 | 0 | 0.0216 | 0.0980 |
| 43 | 488.0 | 43 | 0 | 0.0213 | 0.0954 |
| 44 | 493.1 | 54 | 0 | 0.0216 | 0.0941 |
| 43 | 494.5 | 158 | 0 | 0.0214 | 0.0909 |
| 38 | 489.8 | 33 | 0 | 0.0205 | 0.0898 |
| 46 | 493.3 | 50 | 0 | 0.0219 | 0.0898 |
| 49 | 499.9 | 37 | 0 | 0.0225 | 0.0889 |
| 46 | 490.1 | 24 | 0 | 0.0219 | 0.0887 |

## Appendix C: Additional Tables and Figures

Figures 9 and 10 show the distribution of the bootstrap ATE estimates and of the placebo estimates.

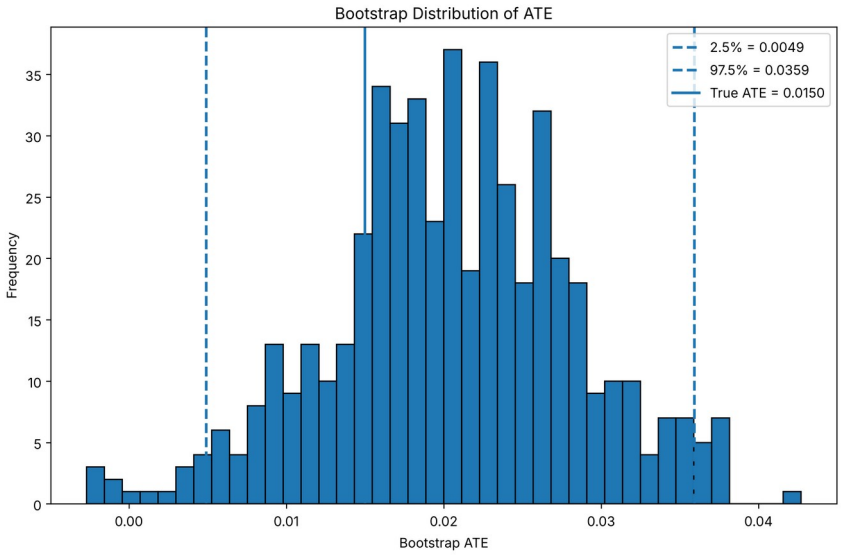



**Figure 9.** Bootstrap distribution of the ATE.

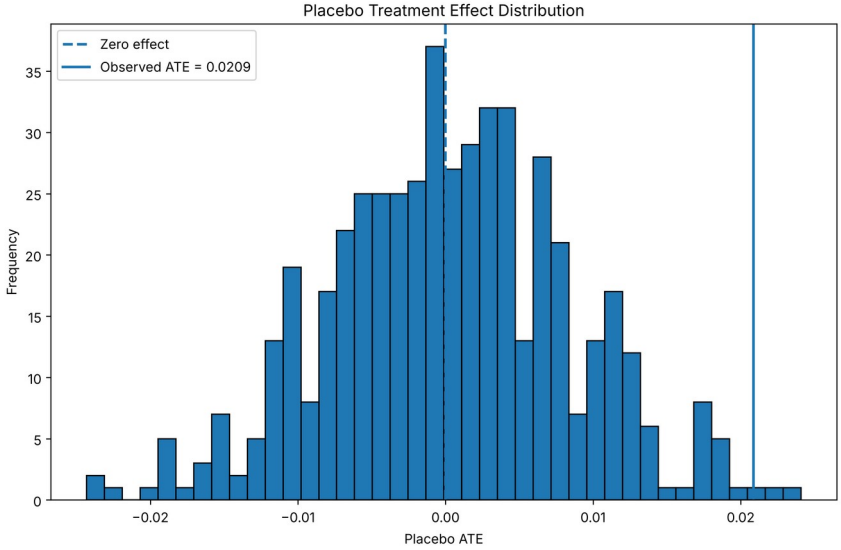



**Figure 10.** Placebo distribution of the estimated effect.